

C++ Programming Textbook Notes

Module: Dynamic Memory Allocation | new & delete Operators, Objects on Heap & Arrays

Author: Gaurav Kandpal

Platform: CS-Simplified

Level: Beginner to Advanced

1. DYNAMIC MEMORY ALLOCATION: CONCEPTUAL FOUNDATIONS

Static/Automatic Memory vs. Dynamic Memory

In C++, memory allocated for standard variables (such as `int x;` inside a function) has automatic storage duration on the **Call Stack**. The size of these variables must be known at compile time, and their memory is automatically deallocated when the enclosing scope terminates.

However, real-world applications often require memory where the exact size depends on runtime input (such as reading variable numbers of records) or data whose lifetime must persist across function calls. For this purpose, modern operating systems provide the **Heap (Free Store)**.

What are new and delete?

- **new Operator:** A C++ memory allocation operator used to dynamically allocate storage blocks on the Heap at runtime. It computes the required byte size automatically based on the data type, initializes the memory (invoking constructors for class objects), and returns a typed memory address pointer pointing to that block.
- **delete Operator:** A C++ memory deallocation operator used to release dynamically allocated Heap memory previously obtained via `new`, returning that memory back to the Operating System. For class objects, it invokes the object's destructor immediately before reclaiming memory.

How new and delete Work in Memory

When executing `int* ptr = new int;;`

1. A pointer variable `ptr` is allocated on the **Call Stack** (e.g., 8 bytes on a 64-bit architecture).
2. The `new` operator requests 4 bytes of contiguous memory from the **Heap**.
3. The Heap memory address (e.g., `0x00508000`) is returned and assigned into `ptr`.
4. When `delete ptr;` executes, the Heap memory at `0x00508000` is freed. The stack pointer `ptr` remains on the stack until its scope exits, but should be reset to `nullptr` to avoid becoming a *Dangling Pointer*.

[DYNAMIC MEMORY ALLOCATION IN RAM: STACK VS. HEAP]

Code Statement: `int* ptr = new int; *ptr = 42;`

STACK MEMORY (Local Pointer):

Pointer 'ptr' (8 Bytes)
Address: 0x7ffd10
Holds Value: [0x00508000]

HEAP MEMORY (Free Store):

Dynamic Int Block (4 Bytes)
Address: 0x00508000
Value Stored: [42]

—points to—→

delete ptr; frees this block

2. SYNTAX & ESSENTIAL RULES FOR NEW AND DELETE

1. Single Variable Allocation

```
// Allocation
data_type* ptr = new data_type;           // Uninitialized primitive
data_type* ptr = new data_type(value);    // Direct initialization

// Deallocation
delete ptr;
ptr = nullptr;                           // Best practice: reset dangling pointer
```

2. Dynamic Object Allocation

When `new ClassName(arguments);` is invoked, two steps occur automatically:

1. Memory block on Heap is allocated for all instance attributes.
2. The constructor `ClassName(arguments)` is invoked immediately to initialize those attributes.

3. Dynamic Array Allocation

When allocating contiguous blocks of memory for arrays, use the bracket syntax:

```
// Array Allocation & Deallocation
int* arr = new int[size]; // Allocates array of 'size' elements
delete[] arr;              // MANDATORY: Must use delete[] for arrays!
```

Critical Rule: Pairing delete and delete[]

Never use `delete ptr;` on memory allocated with `new[]`! Using single `delete` on an array invokes the destructor only for the first element and corrupts Heap management meta-data, leading to undefined behavior or segmentation faults.

3. NEW / DELETE VS. C-STYLE MALLOC() / FREE()

Feature	C-Style (malloc / free)	Modern C++ (new / delete)
Nature	Library functions (declared in <code><cstdlib></code>).	Built-in C++ language operators .
Constructor / Destructor	Does not call constructors or destructors.	Automatically calls constructors on <code>new</code> and destructors on <code>delete</code> .
Type Safety & Casting	Returns <code>void*</code> ; requires explicit type casting.	Returns exact typed pointer (e.g., <code>int*</code>); type-safe.
Size Calculation	Manual calculation via <code>sizeof(type) * n</code> .	Automatic calculation by compiler.
Failure Handling	Returns <code>NULL</code> upon memory exhaustion.	Throws <code>std::bad_alloc</code> exception (or returns <code>nullptr</code> with <code>nothrow</code>).
Array Deallocation	Same <code>free(ptr)</code> used for singles and arrays.	Requires distinct <code>delete[]</code> syntax for arrays.

4. PRACTICAL CODE DEMONSTRATIONS

Example 1: Allocating a Single Primitive Variable

```
// Allocating a Single Variable Dynamically
#include <iostream>
using namespace std;

int main() {
    // 1. Allocate memory on Heap for an integer and store address in 'ptr'
    int* ptr = new int;

    // 2. Assign value at that allocated heap address
    *ptr = 42;

    cout << "Allocated Heap Address: " << ptr << endl;
    cout << "Value at Address (*ptr): " << *ptr << endl;

    // 3. Deallocate memory to prevent memory leaks
    delete ptr;
    ptr = nullptr; // Nullify pointer

    return 0;
}
```

Example 2: Allocating an Object Dynamically on the Heap

```
// Allocating a Class Object Dynamically using new
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
    string name;
    int age;

    // Parameterized Constructor
    Student(string n, int a) {
        name = n;
        age = a;
        cout << "Constructor: Allocated and initialized Student object." << endl;
    }

    // Destructor
    ~Student() {
        cout << "Destructor: Student object cleaned up." << endl;
    }

    void display() {
        cout << "Name: " << name << " | Age: " << age << endl;
    }
};

int main() {
    // Step 1: new allocates memory block in Heap for attributes 'name' and 'age'
    // Step 2: Student("Alice", 20) constructor is executed automatically
    Student* s1 = new Student("Alice", 20);

    // Accessing member functions via arrow (->) operator
    s1->display();

    // delete invokes ~Student() and frees heap memory
    delete s1;
    s1 = nullptr;

    return 0;
}
```

5. PITFALLS IN DYNAMIC MEMORY MANAGEMENT

1. Memory Leak

Occurs when Heap memory is allocated via new but never freed using delete before the pointer loses its reference. The memory stays occupied until the operating system shuts down the process.

2. Dangling Pointer

Occurs when delete ptr; is executed, but ptr is not reset to nullptr. The pointer continues pointing to deallocated, invalid memory. Dereferencing it results in severe crashes or undefined behavior.

3. Double Free Error

Attempting to call `delete ptr;` twice on the same memory address corrupts internal Heap allocator data structures, immediately crashing the program. Always set `ptr = nullptr;` after deletion, as deleting a null pointer is guaranteed safe in C++.

Best Practice: Modern C++ Resource Management

While manual `new` and `delete` are fundamental to understanding computer science and low-level data structures, modern C++ (C++11 and beyond) strongly encourages using **Smart Pointers** (`std::unique_ptr`, `std::shared_ptr`) and standard containers (`std::vector`, `std::string`) to achieve automatic memory management via RAII.